第 10 讲：图、DFS 与 BFS——在关系网络中搜索

0. 本讲的核心问题

树中的每个节点通常只有一个父节点；图则允许更一般的关系：

- 城市之间有道路；
- 用户之间有关注关系；
- 课程之间有先修关系；
- 网格中的格子彼此相邻；
- 一个状态可以通过若干操作转移到其他状态。

图算法的共同问题是：

> 从一个状态出发，怎样系统地找到能够到达的其他状态，并避免无限重复？

本讲重点不是记大量图论名词，而是建立统一模型：

---

1. 图的基本概念

图由节点和边组成，常记为：

```
G = (V, E)
```

- `V`：节点集合；
- `E`：边集合。

1.1 无向图

边没有方向：

```
A — B
```

表示 A 能到 B，B 也能到 A。道路双向通行时可以建模为无向图。

1.2 有向图

边有方向：

```
A → B
```

表示可以从 A 到 B，不一定能从 B 到 A。课程先修关系和网页链接常是有向图。

1.3 加权图与无权图

无权图只关心是否有边，或把每条边都看作代价 1。

加权图的边带有距离、费用、时间等代价：

```
A --5--> B
```

普通 BFS 只直接保证无权图或等权图的最短边数。边权不同的最短路需要其他算法，例如 Dijkstra。

1.4 路径、环与连通分量

- 路径：沿边经过的一系列节点；
- 环：从某节点出发，沿边最终回到它；
- 连通分量：无向图中彼此可以到达的一组节点。

---

2. 图怎样存入代码

2.1 邻接表

```python
graph = {
    "A": ["B", "C"],
    "B": ["A", "D"],
    "C": ["A", "D"],
    "D": ["B", "C"],
}
```

`graph[node]` 保存从 `node` 能直接到达的邻居。

邻接表适合大多数 LeetCode 图题。如果有 `V` 个节点、`E` 条边：

```
O(V + E)
```

无向图的一条边通常在邻接表中出现两次：

```python
graph[a].append(b)
graph[b].append(a)
```

2.2 邻接矩阵

用二维数组记录任意两点之间是否有边：

```python
matrix[i][j] = 1
```

访问一条特定边是 `O(1)`，但无论边多不多都需要 `O(V²)` 空间。节点很多、边很少时通常不如邻接表。

2.3 边列表

```python
edges = [
    ("A", "B"),
    ("A", "C"),
    ("B", "D"),
]
```

题目经常以边列表给出输入，需要先转成邻接表。

```python
def build_undirected_graph(n, edges):
    graph = [[] for _ in range(n)]

    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)
```

```
    return graph
```

2.4 网格也是图

网格中的可通行格子是节点；上下左右的合法移动是边。

```
[python]
directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
```

很多看似没有“图”的题，本质上仍是在隐式图中搜索。

---

3. DFS：沿一条路尽量深入

深度优先搜索 DFS 的策略是：

> 选择一个尚未访问的邻居，沿它继续深入；走不下去时再返回。

可以用递归，也可以用显式栈。

3.1 递归 DFS 模板

```
[python]
def dfs(graph, node, visited):
    visited.add(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
```

调用：

```
[python]
visited = set()
dfs(graph, start, visited)
```

递归函数的含义应该能说成一句话：

> `dfs(node)` 访问 `node` 能够到达、且此前未访问的全部节点。

3.2 为什么先标记再递归

无向图中 A 的邻居有 B，B 的邻居又有 A：

```
A — B
```

如果进入 A 后不立即标记，A 调用 B，B 又调用 A，会无限递归。

因此通常遵守：

```
→      →
```

3.3 显式栈 DFS

```
[python]
def dfs_iterative(graph, start):
    stack = [start]
    visited = {start}

    while stack:
        node = stack.pop()

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                stack.append(neighbor)

    return visited
```

这与递归 DFS 使用相同思想，只是把系统调用栈换成自己维护的 `stack`。

---

4. BFS：按距离一层层扩展

BFS 使用队列：

```
[python]
from collections import deque

def bfs(graph, start):
    queue = deque([start])
    visited = {start}

    while queue:
        node = queue.popleft()

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return visited
```

BFS 与第 5 讲相同：入队时标记，避免同一个节点被多个前驱重复加入队列。

4.1 DFS 与 BFS 如何选择

目标	更自然的方法
判断是否可达	DFS 或 BFS
遍历整个连通块	DFS 或 BFS
无权图最少边数	BFS
枚举路径、回溯选择	DFS
层序、同时扩散	BFS
递归描述子问题	DFS

很多题两者都能完成。选择你更容易维护状态和证明正确性的写法。

---

5. 图搜索的统一模板

无论 DFS 还是 BFS，都要回答四个问题：

1.
2.
3.
4.

抽象模板：

```
worklist = [start]
visited = {start}

while worklist:
    state = remove_one(worklist)

    if is_goal(state):
        return answer

    for next_state in neighbors(state):
        if next_state not in visited:
            visited.add(next_state)
            add(worklist, next_state)
```

- `worklist` 是栈：DFS；
- `worklist` 是队列：BFS；
- `worklist` 是堆：按优先级搜索。

理解这点后，很多算法不再是互不相关的模板。

---

6. 核心题：岛屿数量

网格中：

- `"1"` 表示陆地；
- `"0"` 表示水；
- 上下左右相邻陆地属于同一座岛。

思路：

1. 扫描每个格子；
2. 如果遇到尚未访问的陆地，发现一座新岛；
3. 从这里 DFS，把整座岛全部标记；
4. 继续扫描。

```
[python]
def num_islands(grid):
    if not grid or not grid[0]:
        return 0

    rows = len(grid)
    cols = len(grid[0])
    visited = set()
    islands = 0
    directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    def dfs(row, col):
        visited.add((row, col))
```

```
            for dr, dc in directions:
                nr = row + dr
                nc = col + dc

                if not (0 <= nr < rows and 0 <= nc < cols):
                    continue
                if grid[nr][nc] != "1":
                    continue
                if (nr, nc) in visited:
                    continue

                dfs(nr, nc)

        for row in range(rows):
            for col in range(cols):
                if grid[row][col] == "1" and (row, col) not in visited:
                    islands += 1
                    dfs(row, col)

        return islands
```

6.1 为什么岛屿数在启动 DFS 时增加

每次从未访问陆地启动 DFS，说明此前的所有岛屿搜索都没有到达它，因此它属于一个新连通分量。

DFS 内部访问到的其他陆地与起点连通，不应重复计数。

6.2 使用网格本身标记

如果允许修改输入，可以把访问过的陆地改为水：

```
[python]
grid[row][col] = "0"
```

这样不需要额外 `visited`，但会改变输入。

复杂度：

```
O(rows × cols)
O(rows × cols)   # visited
```

---

7. 连通分量数量

给定 `n` 个节点和无向边，求有多少个彼此独立的连通分量。

```
[python]
def count_components(n, edges):
    graph = [[] for _ in range(n)]

    for a, b in edges:
        graph[a].append(b)
        graph[b].append(a)

    visited = set()
    components = 0

    for start in range(n):
        if start in visited:
            continue

        components += 1
        stack = [start]
        visited.add(start)

        while stack:
            node = stack.pop()
```

```
                for neighbor in graph[node]:
                    if neighbor not in visited:
                        visited.add(neighbor)
                        stack.append(neighbor)

        return components
```

这个模式与岛屿数量完全相同：

```
                →       +1 →
```

---

8. 图中的环

8.1 无向图判断环

在无向图 DFS 中，从 A 走到 B 后，B 一定能看到 A。但这条“回到父节点”的边不代表环。

因此递归时保存父节点：

```
[python]
def has_cycle_undirected(graph):
    visited = set()

    def dfs(node, parent):
        visited.add(node)

        for neighbor in graph[node]:
            if neighbor == parent:
                continue

            if neighbor in visited:
                return True

            if dfs(neighbor, node):
                return True

        return False

    for node in graph:
        if node not in visited and dfs(node, None):
            return True

    return False
```

8.2 有向图为什么更复杂

有向图中，指向“曾经访问过的节点”不一定形成环。只有指向当前递归路径上的节点，才说明出现回边。

可以使用三种状态：

```
0
1
2
```

```
[python]
def has_cycle_directed(graph):
    state = {node: 0 for node in graph}

    def dfs(node):
        if state[node] == 1:
            return True
        if state[node] == 2:
            return False

        state[node] = 1
```

```
                for neighbor in graph[node]:
                    if dfs(neighbor):
                        return True

                state[node] = 2
                return False

            for node in graph:
                if state[node] == 0 and dfs(node):
                    return True

            return False
```

`state == 1` 可以理解为递归栈中的节点。

---

9. 拓扑排序：处理先修关系

课程依赖：

```
A     B A → B
```

如果依赖中有环：

```
A → B → C → A
```

就无法完成所有课程。

拓扑排序为有向无环图产生一个满足所有先后关系的顺序。

9.1 入度

节点的入度是指向它的边数。

入度为 0 表示当前没有未完成的先修要求，可以立即处理。

9.2 Kahn 算法：基于 BFS

```
[python]
from collections import deque

def can_finish(num_courses, prerequisites):
    graph = [[] for _ in range(num_courses)]
    indegree = [0] * num_courses

    for course, prerequisite in prerequisites:
        graph[prerequisite].append(course)
        indegree[course] += 1

    queue = deque()

    for course in range(num_courses):
        if indegree[course] == 0:
            queue.append(course)

    completed = 0

    while queue:
        course = queue.popleft()
        completed += 1

        for next_course in graph[course]:
            indegree[next_course] -= 1

            if indegree[next_course] == 0:
                queue.append(next_course)

    return completed == num_courses
```

不变量：

> 队列中都是当前所有先修要求已经满足的课程。

如果最后处理数量小于课程总数，剩余节点被环互相依赖，没有任何一个能达到入度 0。

时间和空间均为 `O(V + E)`。

---

10. 克隆图：旧节点到新节点的映射

克隆图不能只复制节点值，还要复制连接关系，并保证每个旧节点只对应一个新节点。

使用字典：

```
      →

[python]
def clone_graph(node):
    if node is None:
        return None

    copies = {}

    def dfs(original):
        if original in copies:
            return copies[original]

        copy = Node(original.val)
        copies[original] = copy

        for neighbor in original.neighbors:
            copy.neighbors.append(dfs(neighbor))
```

```
            return copy

        return dfs(node)
```

为什么创建副本后要立即写入 `copies`，再递归邻居？

因为图可能有环。如果等邻居全部完成后再记录，沿环递归时会再次复制同一节点并无限递归。

`copies` 同时承担：

- visited；
- 旧节点到新节点的映射。

---

11. 递归深度与迭代写法

Python 的递归深度有限。一条很长的链状图可能触发：

```
RecursionError
```

面试题通常允许递归解法，但当数据可能很深时，应考虑显式栈。

递归和迭代的核心状态对应关系：

递归	迭代
函数参数	栈元素
调用函数	压栈
函数返回	弹栈
局部变量	栈元素中保存的状态

如果只需访问节点，简单栈即可；如果需要模拟“进入节点”和“退出节点”两个阶段，栈元素还要保存阶段信息。

---

12. 图搜索复杂度

邻接表中：

- 每个节点最多标记一次；
- 每条有向边最多检查一次；
- 无向边在邻接表中出现两次，但仍是常数倍。

因此 DFS 和 BFS 通常都是：

```
O(V + E)
O(V)
```

网格有 `rows × cols` 个格子，每个格子最多四条边：

```
O(rows × cols)
O(rows × cols)
```

不要把嵌套循环自动判断成 `O(n²)`。图搜索中，内层邻居循环在全局加起来只检查所有边。

---

13. 常见错误

13.1 无向边只加入一个方向

无向边 `(a, b)` 需要同时加入 `a → b` 和 `b → a`。

13.2 忘记 visited

图可能存在环，没有访问标记会无限循环。

13.3 标记太晚

BFS 和迭代 DFS 通常在加入工作表时就标记，避免重复安排。

13.4 只从节点 0 搜索

图可能不连通。若题目要求处理整张图，需要外层遍历所有节点。

13.5 混淆有向与无向的环检测

无向图要排除父边；有向图要区分“当前路径中”和“已经处理完”。

13.6 网格先访问再检查边界

必须先确认坐标合法，再读取 `grid[nr][nc]`。

13.7 拓扑图方向建反

若 `[course, prerequisite]` 表示先学 `prerequisite` 才能学 `course`，边应为：

```
prerequisite → course
```

13.8 把节点值当作节点身份

图节点可能值相同。克隆、相交等问题需要使用节点对象本身作为键。

---

14. 本讲检查表

```
1.
2.
3.
4.
5.
6.    DFS    BFS
7. visited
8.
9.
10.
11.
12.
```

15. 本讲总结

1. 图是节点与边构成的通用关系模型。
2. 邻接表是 LeetCode 中最常用的图表示，空间为 `O(V+E)`。
3. DFS 沿路径深入，BFS 按距离分层；两者都必须防止重复访问。
4. 连通分量题使用“外层扫描 + 内层搜完整块”。

5. 有向图环检测需要区分未访问、当前路径中、已经完成三种状态。
6. 拓扑排序不断处理入度为 0 的节点，可判断课程依赖是否可完成。
7. 图搜索复杂度通常为 `O(V+E)`，网格搜索通常为 `O(rows×cols)`。